6、boost库应用

嵌入式设备应用开发（boost库应用）

原创

嵌入式-老费 已于 2023-08-21 08:44:06 修改

阅读量453 收藏

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： linux

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文介绍了如何在Ubuntu上安装和使用Boost库进行嵌入式设备开发，通过三个示例：基本使用、多线程编程和网络程序，展示了Boost库的便捷性和跨平台性，强调其在提高开发效率上的价值。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

嵌入式开发过程中不可避免在很多情况下，需要使用到posix的api函数。一方面，这些api函数确实可以帮助我们解决一些问题；但是另外一方面，因为平台的差异，如果一段时间不做嵌入式开发，那么从windows再切换到liunx的时候，很多的函数又要重新学习一遍，这对我们来说确实很不方便。所以，我们总是希望能够找到一个好用的、跨平台的、高效的函数库，而boost就是这里面的佼佼者。

boost库既可以在windows平台编译，参与上位机开发，也可以在linux平台下面开发和使用，十分方便。为了说明boost库一般是如何使用的，我们今天在linux平台上进行介绍和演示。

1、ubuntu下面安装boost库

安装boost库比较简单，直接apt-get即可。因为很多同学一开始并不熟悉需要安装哪些boost库，所以这部分建议直接安装全部boost内容即可，

```
sudo apt-get install libboost-all-dev
```

2、测试第一个代码demo1.cpp

```
#include<iostream>
#include<boost/bind.hpp>

using namespace std;
using namespace boost;
```

```
int fun(int x,int y)
{
    return x+y;
}

int main()
{
    int m = 10;
    int n = 20;
    cout<<boost::bind(fun,_1,_2)(m,n)<<endl;

    return 0;
}
```

编译的方法也是很简单，即，

```
g++ demo1.cpp -g -o demo1
```

不出意外，就可以生成一个demo1的可执行文件，执行./demo1程序，就可以一个30的打印。这个文件的意义就在于，检验boost库是不是真的安装好了。毕竟如果没有安装好，那么相关的头文件是不会被识别的。

3、测试第二个程序demo2.cpp

```
#include <boost/thread/thread.hpp>
#include <iostream>

#include <unistd.h>

/*
* g++ thread.cpp -lboost_system -lboost_thread - lpthread
*/

void helloA()
{
        std::cout << "I'm thread A !  --- Start " << std::endl;
        sleep(3);
        std::cout << "I'm thread A !  --- OVER " << std::endl;
```

```
}

void helloB()
{
        std::cout << "I'm thread B !  --- Start " << std::endl;
        sleep(3);
        std::cout << "I'm thread B !  --- OVER " << std::endl;

}

int main(int argc, char* argv[])
{
        boost::thread thrdA(&helloA);
        boost::thread thrdB(&helloB);
        thrdA.join();
        thrdB.join();
        return 0;
}
```

这第二个程序是关于多线程编程的。一开始的时候分别启动threadA、threadB，然后等待threadA、threadB执行结束。整个代码比较简洁，表达的意思也很清晰。编译方法如下，

```
g++ demo2.cpp -g -o demo2 -lboost_system -lboost_thread -lpthread
```

执行结束后，就可以看到demo2已经编译好了。这个时候输入./demo2就可以看到这样的打印效果，

```
$ ./demo2
I'm thread B !  --- Start
I'm thread A !  --- Start
I'm thread B !  --- OVER
I'm thread A !  --- OVER
```

4、测试第三个程序，网络程序

boost库里面有一个字库，即boost.asio，是专门处理网络的，使用也很方便。并且boost.asio有很多的网络示例，也不复杂，可以多多练习。这里举的例子来自于boost.asio的这两个链接，

```
https://www.boost.org/doc/libs/1_76_0/doc/html/boost_asio/example/cpp03
/chat/chat_server.cpp
https://www.boost.org/doc/libs/1_76_0/doc/html/boost_asio/example/cpp03
/chat/chat_client.cpp
```

编译的话，直接输入下面的命令，

```
g++ chat_server.cpp -g -o chat_server -lpthread -lboost_thread
g++ chat_client.cpp -g -o chat_client -lpthread -lboost_thread
```

此时需要验证的话，一般是先启动chat_server，即sudo ./chat_server 127.0.0.1 20000。启动好了chat_server，可以继续输入chat_client，即./chat_client 127.0.0.1 20000，那么就可以在chat_client输入字符了。一般输入什么，就会在console上回显什么，类似于一个echo功能。

5、总结

本身boost的功能很多，远远不止上面说的这些。大家在嵌入式开发的时候，尽量多多使用boost这样的开发库，可以大大提高开发效率，把主要的精力放在算法、或者是业务逻辑上去。